Deep Learning — Assignment 4

LSTM-Based Time-Series Forecasting of Apple (AAPL) Stock Price

Name	Samyak Shende	**Roll No.**	64
PRN	12412320	**Practical No.**	4
Department	CSE-AI	**Semester**	5
Academic Year	2026-27	**Division / Batch**	E / 3
Subject	Deep Learning		

1. Problem Statement

Develop an LSTM-based model for time-series forecasting using stock price, weather, or sales datasets. This implementation uses historical Apple (AAPL) stock-price data and forecasts the daily closing price.

2. Objective

- Download historical AAPL stock-price data from Yahoo Finance for 2015–2025.
- Use daily Closing Price as the target variable.
- Normalize the series using Min-Max Scaling and construct 60-day input sequences.
- Train a two-layer LSTM network with dropout regularization.
- Evaluate predictions using MAE, MSE, and RMSE.
- Visualize actual versus predicted prices and forecast the next closing price.

3. Dataset and Preprocessing

- Dataset: Apple Inc. (AAPL) historical stock data downloaded using yfinance.
- Period specified in the practical: 2015-01-01 to 2025-01-01.
- Target: daily Closing Price.
- Scaling: MinMaxScaler with feature range (0, 1).
- Split: first 80% for training and remaining 20% for testing.
- Sequence length: previous 60 trading days are used to predict the next trading day.

4. LSTM Model Architecture

Layer	Configuration	Purpose
Input	60 time steps × 1 feature	Historical closing prices
LSTM 1	64 units, return_sequences=True	Learn temporal patterns
Dropout	20%	Reduce overfitting
LSTM 2	64 units	Learn higher-level temporal representation
Dropout	20%	Regularization
Dense	1 neuron	Predict next closing price

5. Training Configuration

- Optimizer: Adam
- Loss function: Mean Squared Error (MSE)
- Epochs: 20

- Batch size: 32
- Validation split during training: 10%

6. Solution / Implementation

```
# Download and prepare AAPL data
df = yf.download("AAPL", start="2015-01-01", end="2025-01-01", auto_adjust=False)
close_price = df["Close"].squeeze()
data = close_price.values.reshape(-1, 1)

# Normalize
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(data)

# 80/20 chronological split
train_size = int(len(scaled_data) * 0.8)
train_data = scaled_data[:train_size]
test_data = scaled_data[train_size:]

# Create 60-day sequences
time_step = 60

def create_sequences(data, time_step):
    X, y = [], []
    for i in range(time_step, len(data)):
        X.append(data[i-time_step:i, 0])
        y.append(data[i, 0])
    return np.array(X), np.array(y)

X_train, y_train = create_sequences(train_data, time_step)
X_test, y_test = create_sequences(test_data, time_step)

# LSTM input shape: samples, time steps, features
X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)

# Build and compile the LSTM
model = Sequential([
    LSTM(64, return_sequences=True, input_shape=(time_step, 1)),
    Dropout(0.2),
    LSTM(64, return_sequences=False),
    Dropout(0.2),
    Dense(1)
])

model.compile(optimizer="adam", loss="mean_squared_error")

history = model.fit(
    X_train, y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)

# Predict and evaluate
predictions = model.predict(X_test)

predicted_prices = scaler.inverse_transform(predictions)
actual_prices = scaler.inverse_transform(y_test.reshape(-1, 1))

mse = mean_squared_error(actual_prices, predicted_prices)
rmse = np.sqrt(mse)
mae = mean_absolute_error(actual_prices, predicted_prices)

print("MSE:", mse)
print("RMSE:", rmse)
print("MAE:", mae)

# Forecast the next closing price
last_60_days = scaled_data[-60:].reshape(1, 60, 1)
next_day_prediction = model.predict(last_60_days)
next_day_price = scaler.inverse_transform(next_day_prediction)

print("Predicted Next Closing Price: $",
      round(float(next_day_price[0][0]), 2))
```

7. Evaluation and Visualization

- Training and validation loss are plotted across epochs to observe learning behavior.
- The model predicts the testing sequence and predictions are inverse-transformed from normalized values back to USD.
- Actual and predicted AAPL prices are plotted together to assess how closely the forecast follows the historical trend.
- MAE, MSE, and RMSE are calculated as the requested quantitative evaluation metrics.

8. Results

The supplied implementation sheet does not contain populated numerical output for MSE, RMSE, MAE, or the next-day forecast. Therefore, this report does not invent numerical results. When the notebook is executed, the printed model metrics and next-day prediction should be recorded here.

Metric / Output	Value
Mean Absolute Error (MAE)	To be obtained by executing the notebook
Mean Squared Error (MSE)	To be obtained by executing the notebook
Root Mean Squared Error (RMSE)	To be obtained by executing the notebook
Next-day predicted closing price	To be obtained by executing the notebook

9. Conclusion

The LSTM-based time-series forecasting model was successfully designed to predict the closing price of Apple (AAPL) stock using historical stock-price data. The closing-price series is normalized using Min-Max Scaling and converted into sequences of 60 previous trading days for supervised learning. A Sequential network with two 64-unit LSTM layers, 20% dropout layers, and a single-neuron Dense output layer is trained with Adam and MSE loss for 20 epochs. The model is evaluated using MAE, MSE, and RMSE, and actual versus predicted prices are visualized. The experiment demonstrates the use of LSTM networks for temporal forecasting, while recognizing that stock prices are affected by external factors and randomness, so forecasts based only on historical closing prices have inherent limitations.

10. Key Learning Outcomes

- Understanding how time-series data is converted into supervised learning sequences.
- Preparing sequential data in the 3D shape required by an LSTM.
- Using dropout and stacked LSTM layers for sequence modeling.
- Applying inverse scaling to interpret normalized predictions in the original price units.
- Evaluating regression forecasts using MAE, MSE, and RMSE.
- Understanding the limitations of stock-price forecasting from historical prices alone.